

# Cutroom: Technical Requirements

## Overview and approach

Cutroom is a responsive single page web app. There is no real model inference. Generation is simulated by a mock engine that reproduces queueing, latency, cost and occasional failure.

The framing that matters for this build: because the model is faked, engineering effort is not spent on inference. It is spent on the three things that actually make the product feel real. First, a clean state model for takes and branches. Second, a generation service designed as if a real backend sat behind it. Third, interaction and motion polish. This mirrors the product thesis directly. The model is a commodity, so the plumbing and the feel are the work.

The mock engine sits behind a narrow interface, the same interface a real routing backend would expose. Replacing the stub with real models later would touch one module and leave the UI untouched. That is the point the architecture is meant to demonstrate: production shaped plumbing, not a fake screen.

## Stack

Chosen for speed of building and quality of feel, and aligned with a React and TypeScript workflow.

- **React 18 with TypeScript**, built with Vite.
- **Tailwind CSS** for styling.
- **Zustand** for state. Light enough for a one day build, structured enough for the branching take tree.
- **Framer Motion** for transitions and the feel of iteration. This is where the motion sensibility shows.
- **lucide react** for icons.
- **Local placeholder video and image assets**, bundled in, standing in for generated output.

No backend, no database, no authentication. Optional localStorage keeps the current session so a refresh does not wipe the take tree.

# Architecture

Three layers, kept deliberately separate. The call direction only ever runs one way: UI to store to engine.

1. **The generation engine.** A single module exposing a narrow asynchronous interface: `generateDraft`, `renderFinal`, `remix` and `retry`. Internally it simulates a backend. It queues a job, moves it through its states, and resolves with media or an error. Nothing else in the app knows the engine is a stub. This module embodies the thesis and is the place a reviewer should look to see systems thinking.
2. **The store.** A Zustand store holding the project, the take tree, the active selection and the simulated credit balance. It owns every transition between takes.
3. **The UI.** Presentational React components: the prompt and intent panel, the draft grid, the render view, the version rail and the recovery surface. Components read from the store and reach the engine only through the store, never directly.

The rule that keeps the seam clean: the UI never talks to the engine directly, and the engine never knows the UI exists. Swapping the stub for a real routing backend later touches the engine module alone.

## Data model

The branching take tree is the heart of the model. History and branching both fall out of a single structure.

```
type TakeStatus = 'queued' | 'generating' | 'ready' | 'failed';
type TakeKind = 'draft' | 'render';

interface Intent {
  kind: 'social' | 'ad' | 'cinematic';
  subject: string;
  style: string;
  motion: string;
  mood: string;
}

interface Take {
  id: string;
  parentId: string | null; // null for a root take, set for a branch or
  variant
  kind: TakeKind;
```

```

    status: TakeStatus;
    prompt: string;
    intent: Intent;
    assetUrl?: string;          // present when ready
    error?: string;             // present when failed
    cost: number;               // credits, shown before commit
    createdAt: number;
  }

  interface Project {
    id: string;
    title: string;
    takes: Take[];              // a flat list that forms a tree via parentId
    activeTakeId: string | null;
    credits: number;            // simulated balance
    createdAt: number;
  }

```

The version rail is derived by walking parentId links, so there is no separate history log to keep in sync.

## Mock generation pipeline

The engine simulates a real generation job so the experience is honest about timing and failure.

- **generateDraft:** create four takes in the queued state, move them to generating, then resolve each to ready after a short randomized delay of roughly one to three seconds. Drafts are cheap or free and use low resolution placeholder clips.
- **renderFinal:** take the selected draft, create a render take, run a longer delay of roughly five to ten seconds with visible progress, and charge simulated credits.
- **remix:** branch four variant takes from the selected take with slight prompt or seed variation.
- **Failure injection:** a small random share of jobs, around one in eight, resolve to failed. This is not a bug. Failure has to be real for recovery to be a real feature.
- **Cost model:** a simple function of kind and intent. Drafts sit near zero, renders cost a few credits, and the number is surfaced to the UI before the action.

## State management

The store owns every transition. Core actions:

- **submitDraft(prompt, intent):** calls the engine, inserts the four draft takes, and updates their status as the engine reports back.
- **selectTake(id):** sets the active take, which drives the render view and the rail.
- **render():** commits the active draft, inserts the render take, and deducts credits on success.
- **remix(id):** branches variants from a take.
- **retry(id):** rerolls a failed take at no cost.
- **applyNudge(id, nudge):** branches a new take with one adjusted parameter, for example slower motion or a stronger cinematic style.

Updates are optimistic throughout. Takes appear immediately in a queued or generating state and then settle. The UI is always responsive and never blocked on the engine.

## Interaction, motion and error recovery

This is where the extra hours go and where most clones fall short.

### Motion and feel

- Drafts reveal with a staggered animation as they settle from generating to ready, so a batch feels alive instead of popping in all at once.
- Generating takes show a real skeleton and progress, never a spinner on a blank screen.
- Selecting a draft animates it into the render view, and the version rail slides while keeping the branch structure visible.
- Every control shows its state clearly: hover, active, disabled and cost.
- The first load is a designed empty state that points the user at the one thing to do, not a blank canvas.
- Small, quick micro feedback on the satisfying actions: picking a take, spinning variants, a successful render. Present, never in the way.

### Error recovery

- A failed take renders in place with a clear, human explanation and two actions: reroll for free, or adjust and retry.

- Corrective nudges are preset chips such as too fast, wrong character, more cinematic or less busy. Each maps to a concrete parameter change and branches a new take, so the user fixes by tapping rather than rewriting a prompt.
- A reroll after a failure never costs credits. This is a deliberate trust decision and should be visible in the interface.
- Because recovery branches rather than replaces, both the failed take and the fixed take stay on the rail, and the user never loses an earlier good state.

The goal is that iterating feels like play. Someone who edits video or has built games will feel the difference, and so will the reviewer.

## Build plan and technical out of scope

**The twenty four hour budget.** The extra time over a nine hour build goes into depth and feel, not more screens.

- **Hours 0 to 3:** project setup, types, the store skeleton, and the mock engine interface with fake timing working end to end.
- **Hours 3 to 9:** the core flow wired up. Prompt and intent panel, draft grid, selection, render view. Functional but rough.
- **Hours 9 to 14:** the iteration and recovery loop. Variation, branching, the version rail, failure injection and the recovery surface.
- **Hours 14 to 20:** feel and motion. Transitions, skeletons, staggered reveals, empty and hover states, micro feedback. This is the differentiator, so it gets real time.
- **Hours 20 to 22:** prompt assist and the command palette if the earlier phases held, otherwise more polish.
- **Hours 22 to 24:** the decisions writeup and a two minute screen recording that walks through the thesis.

The rule for the whole build: if you are tempted to add a new screen, stop and polish the core flow instead.

### Technical out of scope

- No backend or database. Everything runs client side.
- No real model calls. The engine is a stub behind a real shaped interface.
- No authentication and no real billing.
- No teams or multiplayer.
- No native app.